

壹、前言

一、研究動機

加密技術隨著近期加密貨幣的崛起開始被更加關注，不論駭客或各國商業間諜，無不想在這龐大的貨幣世界中獲得一杯羹，所以加密技術已經成為現代生活中不可或缺的一部份。我從圖解密碼學與資訊安全中深入了解與發現加密技術常常出現在日常生活中，並保護著我們免於被入侵或被竊取資料，但在閱讀相關書籍時發現大多數的書籍僅有介紹理論及歷史發展方面，所以運用功能多元的樹梅派，將一些簡單的密碼學結合硬體及實際情況具現化，除了可以實際運用到加密技術，也讓理論不只空談，更能直接運用在現實生活中。

二、研究目的

以探討及實踐簡單加密技術為出發點，利用樹梅派的高擴充性再結合加入多種功能，同時了解並學習多方面的知識，例如：資料庫的設定與連線、flask 套件或樹梅派結合硬體的運用都會涉略到。希望能藉由這次的實作將書中的知識具現化並使其具有功能性。

三、原理與理論

這次的實做主要的原理是運用 SHA-256 和 RSA 加解密的一些特性來強化資料庫資料的安全性及完成身分驗證的功能。

（一）SHA-256

SHA-256 是一種現今常用的雜湊函數，所謂的雜湊函數簡單說就是能把任意長度的雜湊值轉換成固定長度的摘要。一份安全的雜湊函數必須要能滿足以下幾點：

- 1.只要明文有資料內容的改變，產出的雜湊值必改變
- 2.明文能輕易地經雜湊後形成摘要，但無法從摘要回推回明文
- 3.具備確定性，即相同的明文一定對應到相同的摘要
- 4.能有效阻擋碰撞攻擊（指找出一份內容不同卻可產出相同摘要的明文，藉此使摘要的驗證資料完整性功能失去作用，又稱生日密碼攻擊）而 SHA-256 正是因為目前能滿足而被運用廣泛，而我們這次正是運用上述的特性來保護資料庫中的資料，因為如果直接將資料明文直接存在資料庫中的話，當資料庫資料不幸被竊取或是比對資料時被人從中擷取資料的話，很容易將資料赤裸裸的洩漏出去，甚至能讓不法者能通過比對資料的驗證直接入侵。但如果我們先將資料雜湊後存於資料庫中，就算被他人竊取了也無法被還原成明文，因此資料內容就可以有效地得到保護，更能防止比對資料驗證被破解。

（二）RSA 非對稱加密

非對稱是密碼的特點是將傳統密碼的金鑰 K 拆成兩個部分，加密密鑰 Ke 與解密密鑰 Kd。而其中的演算法能確保不能由 Ke 去回推 Kd，因此即使將 Ke 公布出來也不影響密碼的安全性。依據非對稱式密碼的概念，一個非對稱式密碼能應能滿足下列條件：

- 1.解密演算法 D 能將經由加密演算法 E 計算出的密文還原成明文 M

$$D(E(M, Ke), Kd)$$

- 2.Kd 與 Ke 互換亦成立

$$D(E(M, Kd), Ke)$$

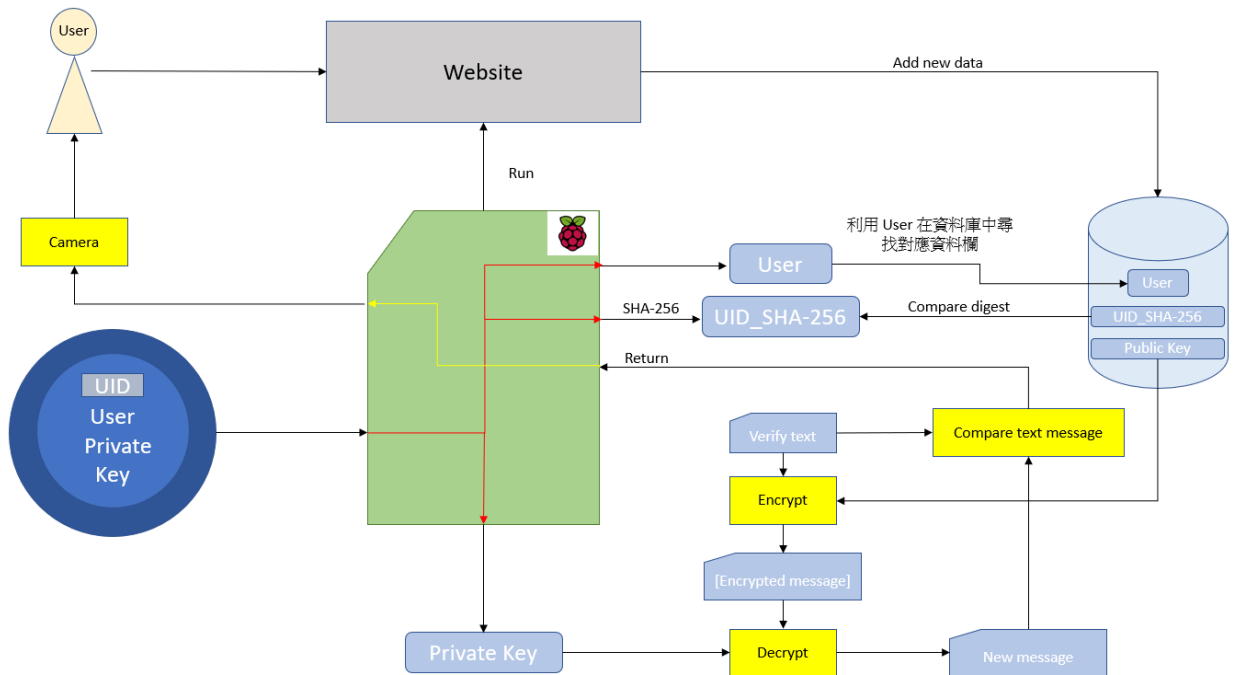
3.無法由 K_e 計算得出 K_d

4.演算法 E 和 D 的運算是高效率的

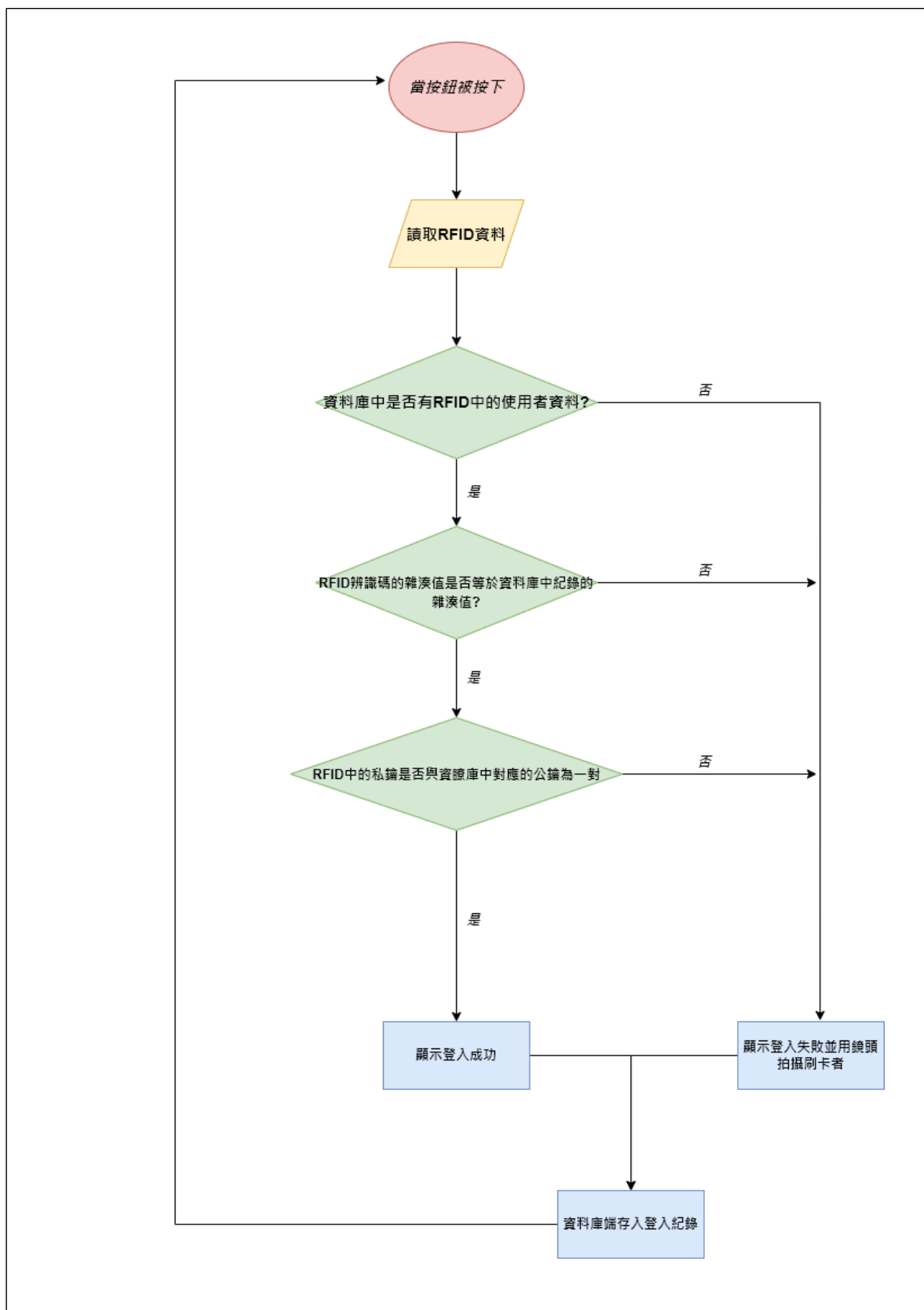
因此我們利用 3.的特性來確保身分驗證的安全性，假設每個使用者都有一個專屬的公鑰存放於資料庫中，而密鑰只有使用者自己知道，再依照 1.，當驗證訊息由資料庫中某位已註冊使用者的公鑰加密後，能被某人以正確的私鑰解密的回原始驗證訊息的話，則代表那位擁有正確對應私鑰的人正是資料庫中紀錄的本人。

四、實驗步驟

利用流程圖說明如何將密碼學技術與樹梅派具現化



圖一 密碼學與樹梅派具現化之關係圖（張晏誠自製）



圖二 密碼學與樹莓派具現化之流程圖（張晏誠自製）

貳、正文

五、實驗方法

(一) 前置作業（資料庫端）

1.在一開始引入伺服器端所需的套件和連線設定檔

```
from flask import Flask , render_template , url_for , request , redirect , Response , jsonify
import config
import hashlib
from flask_bootstrap import Bootstrap
from time import sleep
from datetime import datetime
from flask_sqlalchemy import SQLAlchemy
import pymysql
pymysql.install_as_MySQLdb()
```

圖三 引入插件記錄圖

2.設定好資料庫連線後，分別建立使用者對照表和登入資訊兩表格並設定好表內資料的格式及特性

```
conn = pymysql.Connect(
    host='0.0.0.0',
    port=3306,
    user='root',
    passwd='1021mwmw',
    db='RFID',
    charset='utf8') #配置資料庫連接 (PyMySQL)

app = Flask(__name__)
Bootstrap(app)
app.config.from_object(config)
db = SQLAlchemy(app)

db.init_app(app)
class RFID(db.Model):
    __tablename__ = 'RFID'
    id = db.Column(db.Integer , primary_key = True , autoincrement = True)
    name = db.Column(db.String(20) , nullable = False)
    rfid_code_sha = db.Column(db.String(300) , nullable = False)
    PublicKey = db.Column(db.String(1000), nullable = False)
    n = db.Column(db.String(100), nullable = False)

db.create_all() #建立使用者對照表

class information(db.Model):
    __tablename__ = 'information'
    id = db.Column(db.Integer , primary_key = True , autoincrement = True)
    name = db.Column(db.String(20) , nullable = False)
    RFID_code = db.Column(db.String(50) , default = '[Encrypt]')
    storage_time = db.Column(db.DateTime , default = datetime.now)
db.create_all() #建立登入資訊表格
```

圖四 資料庫連接及控制之程式碼

3.製作能新增使用者資料到資料庫的網站

```
@app.route('/')
def index():
    return render_template('index.html')    #設定網頁主畫面

@app.route('/rfidsetup' , methods=["POST","GET"])
def RFIDsetup():
    if request.method == "POST":
        url1=url_for('index')
        name = request.form["nm"]
        rfid_code = request.form["code"]
        PublicKey = request.form["PK"]
        n = request.form["n"]

        RFID256 = hashlib.sha256(rfid_code.encode("utf-8")).hexdigest()

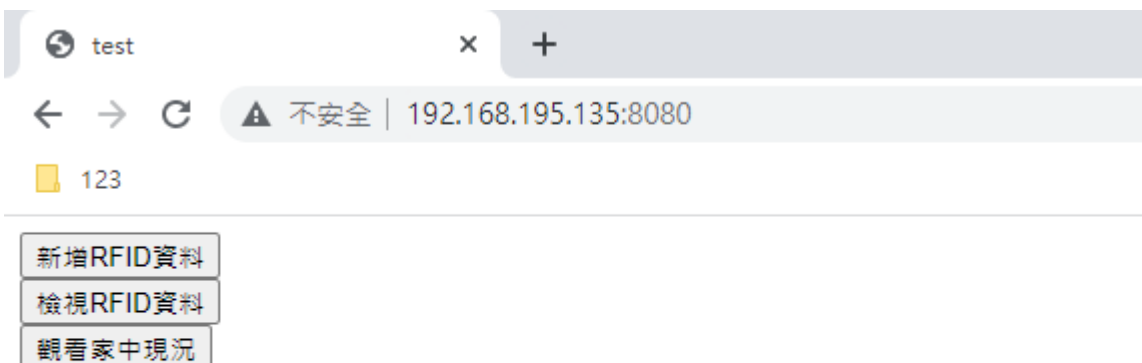
        NewRFID = RFID(name = name , rfid_code_sha = RFID256 , PublicKey = PublicKey, n = n)
        db.session.add(NewRFID)
        db.session.commit()
        return redirect(url1)
    else:
        return render_template('RFIDsetup.html')    #設定輸入使用者的介面
```

圖五 網頁路由設定及資料收集程式碼

4.執行網頁

```
if __name__ == '__main__':
    app.run(debug = True , host = '0.0.0.0' , port = 8080)    #執行網頁
```

圖六 執行網頁程式碼



圖七 網頁成果圖

圖八 新增資料頁面圖

(二) 硬體端的架設

1.先引入所有會用到的插件及連線設定檔，並使用設定檔連線到資料庫端

```
from picamera import PiCamera    #引入鏡頭模組
import time
import sys, math
import RPi.GPIO as GPIO        #啟用樹梅派腳位功能
from mfrc522 import SimpleMFRC522    #引入RFID讀取機的插件
import hashlib                #引入雜湊插件
from time import sleep
import config                #導入資料庫連線設定
from flask import Flask
from flask_sqlalchemy import SQLAlchemy    #用於資料庫連線之功能插件
from datetime import datetime
app = Flask(__name__)
app.config.from_object(config)    #依設定連線資料庫
db = SQLAlchemy(app)
camera = PiCamera()
```

圖九 導入硬體插件與資料庫連線程式碼

2.宣告字符集和驗證訊息，再依序設定硬體端的腳位

```
SYMBOLS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz1234567890 !?.'    #用於RSA加密的部分
message = 'DFGHyRFGHuy65rfghui8OLKJHFCVBHyt456789okjhy65efghTDCVBNMnbfdertgvcdswHGREGShjuyfghjytDCvBnmkIutredVGFRthb'
#用於比對RSA加密前後是否相等的驗證訊息

led = 12
btn = 3    #設定按鈕和燈的腳位

GPIO.setmode(GPIO.BOARD)
GPIO.setup(led, GPIO.OUT)
GPIO.setup(btn, GPIO.IN)
reader = SimpleMFRC522()    #設定硬體腳位的輸入輸出

pre = 0
```

圖十 宣告字符集、驗證訊息及硬體腳位程式碼

3、創建用於 RSA 加密的函式

```
def getBlocksfromText(message, blockSize):
    blockInts = []
    for blockStart in range(0, len(message), blockSize):
        blockInt = 0
        for i in range(blockStart, min(blockStart + blockSize, len(message))):
            blockInt += (SYMBOLS.index(message[i])) * (len(SYMBOLS) ** (i % blockSize))
        blockInts.append(blockInt)
    return blockInts

def getTextfromBlocks(blockInts, messageLength, blockSize):
    message = []
    for blockInt in blockInts:
        blockMessage = []
        for i in range(blockSize - 1, -1, -1):
            if len(message) + i < messageLength:
                charIndex = blockInt // (len(SYMBOLS) ** i)
                blockInt = blockInt % (len(SYMBOLS) ** i)
                blockMessage.insert(0, SYMBOLS[charIndex])
        message.extend(blockMessage)
    return ''.join(message)

def encryptMessage(message, n, e, blockSize):
    encryptedBlocks = []
    for block in getBlocksfromText(message, blockSize):
        encryptedBlocks.append(pow(block, e, n))
    return encryptedBlocks

def decryptMessage(encryptedBlocks, messageLength, n, d, blockSize):
    decryptedBlocks = []
    for block in encryptedBlocks:
        decryptedBlocks.append(pow(block, d, n))
    return getTextfromBlocks(decryptedBlocks, messageLength, blockSize)
```

圖十一 RSA 演算法之函式程式碼

- 4.創建一個迴圈使硬體能重複運行，迴圈內的內容為當按鈕被按下後，點亮 Led 燈並開始讀取 RFID 資訊（識別碼、私鑰和使用者名稱）。讀取完資訊後首先將辨識碼經 SHA-256 雜湊後的值和讀取到的私鑰儲存於變數，方便之後比對資料庫或運算加解密。最後利用剛才讀取到的使用者名稱逐一比對資料庫中的資料後，回傳搜尋結果

```
try:
    while True:
        state = GPIO.input(btn)
        if ((not pre) and state): #當按鈕被按下
            GPIO.output(led, True) #Led燈亮起
            id, text = reader.read() #讀取RFID的資料(識別碼(id)，私鑰和使用者名稱(都在text中並用","分隔))
            id = str(id)
            sha_id = hashlib.sha256(id.encode('utf-8')).hexdigest() #將RFID的識別碼SHA-256雜湊
            inform = text.split(',')
            user = inform[0]
            PrivateKey = int(inform[1])
            sleep(2)
            result = RFID.query.filter(RFID.name == user).first() #利用使用者名稱對照資料庫後得到的結果
```

圖十二 硬體控制程式碼

- 5.如果資料中有對應的資料，則開始進行驗證的動作，動作如下：

- (1)使用資料庫中對應的公鑰將驗證訊息加密，形成密文
- (2) 再用 RFID 中的私鑰進行解密，形成明文
- (3) 使用 if 迴圈來檢查雜湊碼是否與資料庫中資料相同及明文是否與驗證訊息相同
- (4) 如果成功代表身分驗證成功，顯示[Pass]並在資料庫端寫入詳細登入資訊；

反之則顯示[Unknown]並同樣寫入登入資訊，並且用鏡頭拍下入侵者的臉存放於桌面。而如果一開始就無法在資料庫中找到對應資料，則同樣表示身分驗證未通過，與上述之步驟(4)中的驗證失敗處理方式相同。驗證完後，將燈熄滅並等待下次被按下按鈕。最後，清除樹梅派的 GPIO 設定，避免運行同一支程式時跳出頻道被占用的警訊

```
if result != None:    #如果有對應的資料

    PublicKey = int(result.PublicKey)    #抓取資料庫中的公鑰
    n = int(result.n)    #抓取資料庫中的n值
    encryptedMessage = encryptMessage(message , n, PublicKey , 6)    #先使用公鑰RSA加密驗證訊息

    decryptedMessage = decryptMessage(encryptedMessage,300 , n, PrivateKey, 6)
    decryptedMessage = ''.join(decryptedMessage)    #再使用私鑰解密

    if (sha_id == result.rfid_code_sha) and (decryptedMessage == message):    #如果雜湊碼與驗證訊息都比對成功的話，表示身分認證成功
        new = information(name = user)
        db.session.add(new)
        db.session.commit()    #提交登入詳細資訊給資料庫端
        print('Pass')    #顯示登入成功
    else:
        new = information(name = '[Unknown]', RFID_code = id)
        db.session.add(new)
        db.session.commit()    #反之則代表驗證失敗，直接提交登入資訊置資料庫
        pictureName = '-'.join(time.ctime().split())
        camera.capture('/home/yorick/Desktop/%s.jpg' %pictureName)    #拍下一張入侵者的照片並儲存在桌面
        print('Unknown')

    else:
        new = information(name = '[Unknown]', RFID_code = id)
        db.session.add(new)
        db.session.commit()    #沒找到資料也代表驗證失敗，直接提交登入資訊置資料庫
        pictureName = '-'.join(time.ctime().split())
        camera.capture('/home/yorick/Desktop/%s.jpg' %pictureName)    #拍下一張入侵者的照片並儲存在桌面
        print('Unknown')

GPIO.output(led, False)    #將Led燈熄滅
pre = 0
sleep(0.05)

finally:
    GPIO.cleanup()    #清除GPIO設定
```

圖十三 比對資料及硬體動作之程式碼

六、實驗結果

影片連結: https://youtu.be/PFb1f_QPXB4

參、未來展望及改進

七、未來展望

雖然最後成功地把理論與硬體結合實現具現化，但由於這算是從無到有的第一版專案，雖然架構大致上能呈現，但其中還是具有許多淺在問題及缺乏細節，所以這裡提出幾個專案未來需要改進的點

1.安全性

(1) RSA 身分驗證

在這次專案中的 RSA 加解密是我直接依照 RSA 的原理流程做運算，並沒有加入任何額外措施，因此它的安全性是堪憂的，加上寫入 MIFARE 磁卡的空間限制，沒辦法把太長的私鑰寫入其中(一般來說 RSA 的公密鑰至少要 1024 bits 的長度才安全)，所以對於現在的電腦運算速度，只要能成功取的資料庫中的公鑰，要由公鑰在短時間內破解出私鑰是完全有可能的。所以撇除這次是為了使加密理論具現化，如果要在現實生活中實際運用，可能就要優先解決這項缺點。

(2) MIFARE 卡能被任何人讀取訊息

雖然能藉由私鑰來進行身分驗證，但我們存放私鑰的位置很不安全，因為我們在磁卡中保存私鑰時並沒有額外設定任何安全措施來確保我們私鑰的安全，因此磁卡就能被輕易複製，進而破解我們的門禁系統。基於上述兩點對於安全性的疑慮，我希望未來能夠使用 MIFARE 內建的 CRYPTO-1 加密系統來加強，利用 CRYPTO-1 來將磁扣內容加密，雖然 UID 還是無法避免被他人得知，但由於我們的驗證是雙重驗證且已將私鑰用 CRYPTO-1 加密，所以還是能提高安全性讓磁扣卡無法被輕易複製。而至於 RSA 密鑰過短的問題，我目前並沒有直接能解決的想法，但如果換條路走還是能夠解決的，而且方法異常的簡單，就是捨棄 RSA 加密直接將內容由 CRYPTO-1 加密於 MIFARE 磁扣卡中，而驗證的方式則與 UID 驗證一樣，先做雜湊後在與資料庫中的資料作比對。

2.程式的穩定性

由於這次的專案涉及到了較複雜的功能，所以我在程式中使用了一些現成的套件，但有時候套件會因為版本或硬體的不相容，而產生套件無法正常的運行或不相容。而我未來首要改進的問題之一就是由 mfrc522 套件產生有時無法順利感應磁卡的問題，如下

```
^CTraceback (most recent call last):
  File "/home/yourick/Desktop/python_flask/raspberry_web/test.py", line 93, in <module>
    id, text = reader.read() #讀取RFID的資料(識別碼(id)，私鑰和使用者名稱(都在text中並用","分隔))
  File "/usr/local/lib/python3.9/dist-packages/mfrc522/SimpleMFRC522.py", line 19, in read
    id, text = self.read_no_block()
  File "/usr/local/lib/python3.9/dist-packages/mfrc522/SimpleMFRC522.py", line 38, in read_no_block
    (status, TagType) = self.READER.MFRC522_Request(self.READER.PICC_REQIDL)
  File "/usr/local/lib/python3.9/dist-packages/mfrc522/MFRC522.py", line 259, in MFRC522_Request
    (status, backData, backBits) = self.MFRC522_ToCard(self.PCD_TRANSCEIVE, TagType)
  File "/usr/local/lib/python3.9/dist-packages/mfrc522/MFRC522.py", line 219, in MFRC522_ToCard
    if ~(i != 0) and ~(n & 0x01) and ~(n & waitIRQ)):
KeyboardInterrupt

yourick@raspberrypi:~/Desktop/python_flask/raspberry_web $ python test.py
^CTraceback (most recent call last):
  File "/home/yourick/Desktop/python_flask/raspberry_web/test.py", line 93, in <module>
    id, text = reader.read() #讀取RFID的資料(識別碼(id)，私鑰和使用者名稱(都在text中並用","分隔))
  File "/usr/local/lib/python3.9/dist-packages/mfrc522/SimpleMFRC522.py", line 19, in read
    id, text = self.read_no_block()
  File "/usr/local/lib/python3.9/dist-packages/mfrc522/SimpleMFRC522.py", line 38, in read_no_block
    (status, TagType) = self.READER.MFRC522_Request(self.READER.PICC_REQIDL)
  File "/usr/local/lib/python3.9/dist-packages/mfrc522/MFRC522.py", line 259, in MFRC522_Request
    (status, backData, backBits) = self.MFRC522_ToCard(self.PCD_TRANSCEIVE, TagType)
  File "/usr/local/lib/python3.9/dist-packages/mfrc522/MFRC522.py", line 217, in MFRC522_ToCard
    n = self.Read_MFRC522(self.CommIrqReg)
  File "/usr/local/lib/python3.9/dist-packages/mfrc522/MFRC522.py", line 162, in Read_MFRC522
    val = self.spi.xfer2([(addr << 1) & 0x7E) | 0x80, 0])
KeyboardInterrupt
```

圖十四 磁卡感應機報錯圖

由圖中可以得知問題很有可能是出在負責讀寫磁卡的 mfrc522 套件，其中的程式碼可能與樹梅派或是讀卡機產生了些許的不相容，因此常常出現無法順利讀卡的情況。而這個問題我嘗試過重新安裝套件、回滾 SPI(用來讀取 RFID 讀卡器回傳資料的腳位溝通協定)至舊的版本號，也上網找了些資料但是幫助都不大。因此未來我希望能直接藉由檢查套件中的程式編寫來找出問題點，進而去做修改來讓套件能與我所使用的環境完全相容，將讀卡成功的機率拉到九成九。

肆、參考資料

八、參考資料

1. Raspberry Pi 樹梅派：從不懂玩到上手：
陳會安（2017）。旗標出版
2. Python Flask Web 開發入門與項目實戰：
錢游（2019）。機械工業出版
3. Python 密碼學編程：
Al Sweigart（2020）。人民郵電出版